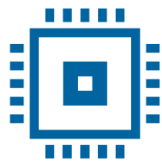


Converter BCD to Digits

Grigore Mihai-Alexandru – inginer verificare

Cilichidreanu Andi-Marian – inginer verificare

Ardeleanu Ștefan-Georgian – documentație tehnică

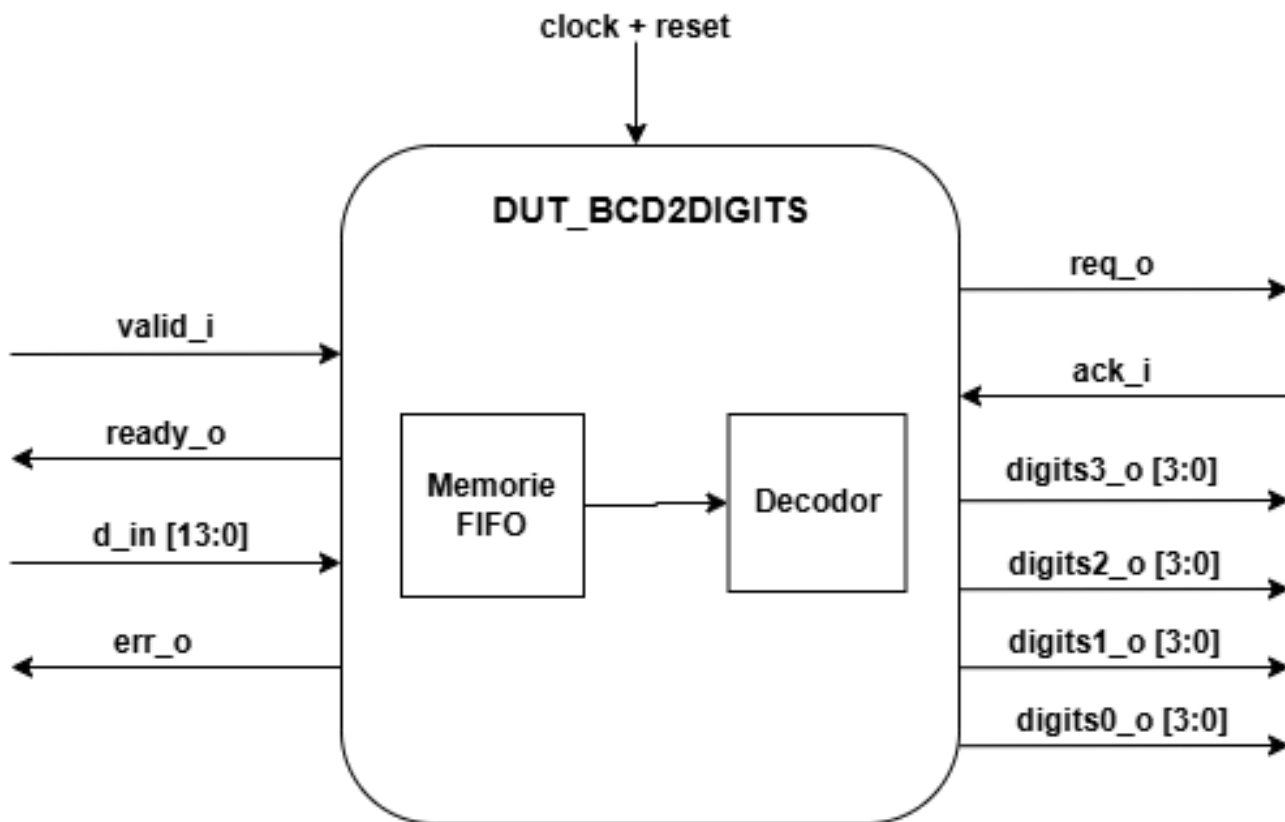


**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**



Diagrama bloc





Sumarul funcţionalităţilor DUT-ului

- Interfaţa de intrare (valid/ready)
 - Date recepţionate – handshake complet între **valid_i/ready_o**;
 - **ready_o = 0** (FIFO plin) => sursa menţine datele stabile până când transferul e posibil;
 - Valori **valide şi procesate** – [0:9999];
 - Valori > 9999 -> invalide şi neprocesate, iar **err_o = 1**

- FIFO intern
 - Configuraţie parametrizabilă (depth/width)
 - Datele sunt citite din FIFO când există date în el, când decodorul nu e ocupat şi când nu avem request activ pe ieşire



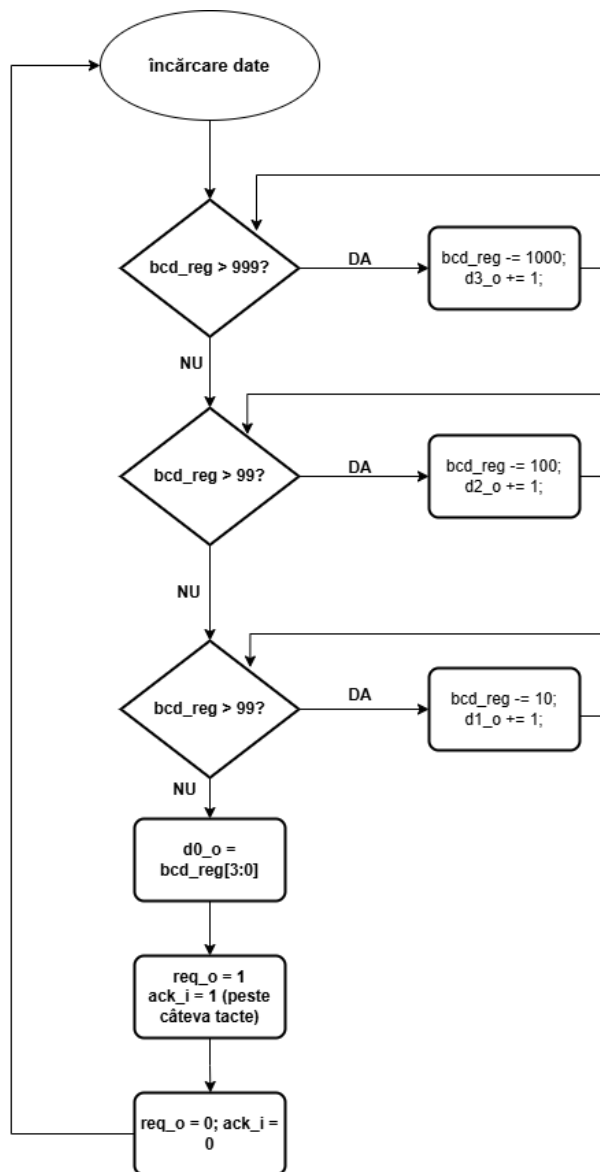
Sumarul funcţionalităţilor DUT-ului

- Algoritmul de conversie BCD – Digits
 - Cifrele (d3 – d0) – determinate prin scăderi succesive şi incrementarea contoarelor corespunzătoare
 - Timp de calcul variabil – maxim de 27 de tacturi pentru 9999
 - Când decodorul este ocupat – $\text{busy_o} = 1$; la terminarea calculului – $\text{req_o} = 1$

- Interfaţa de ieşire (request/acknowledge)
 - Conversie finalizată $\rightarrow \text{req_o} = 1$, iar datele rămân stabile până la recepţionarea acknowledge
 - La primirea confirmării ($\text{ack_i} = 1$), req_o devine inactiv, iar decodorul poate prelua următoarea valoare din FIFO



Diagrama logică a DUT-ului





Porturile DUT-ului

Interfața BCD – intrare

Semnal	Nr. biți	Direcție	Descriere
clk_i	1	Intrare	Semnal de ceas al sistemului
rst_i	1	Intrare	Reset hardware asincron, activ în 1 - resetează FIFO și decodorul
bcd_vld_i	1	Intrare	Semnalizează existența datelor pe magistrală
bcd_din_i [13:0]	14	Intrare	Valoarea binară de convertit (interval fizic 0-16383; valid BCD 0-9999)
bcd_rdy_o	1	Ieșire	DUT-ul poate accepta date; activ când FIFO nu este plin
bcd_err_o	1	Ieșire	Valoarea de intrare depășește 9999 - activ simultan cu bcd_rdy_o=1 și bcd_vld_i=1



Porturile DUT-ului

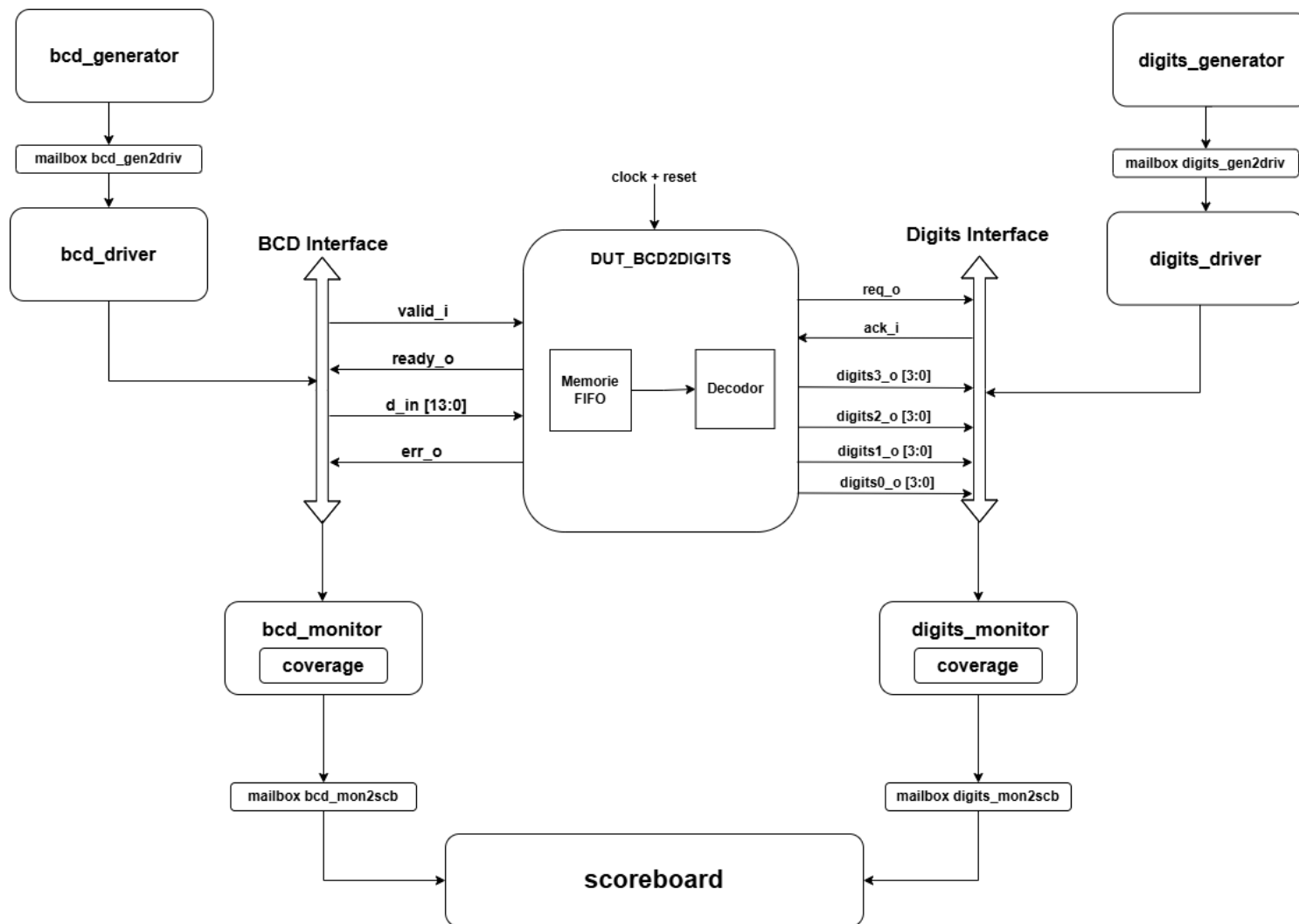
Interfaţa Digits – ieşire

Semnal	Nr. biţi	Direcţie	Descriere
digits_req_o	1	Ieşire	DUT-ul a calculat cifrele şi solicită transferul
digits_ack_i	1	Intrare	Receptorul confirmă preluarea cifrelor
digits_d3_o [3:0]	4	Ieşire	Cifra miilor (valoare 0-9)
digits_d2_o [3:0]	4	Ieşire	Cifra sutelor (valoare 0-9)
digits_d1_o [3:0]	4	Ieşire	Cifra zecilor (valoare 0-9)
digits_d0_o [3:0]	4	Ieşire	Cifra unităţilor (valoare 0-9) - implementată combinaţional (digits_d0_o = bcd_reg[3:0])





Arhitectura mediului de verificare





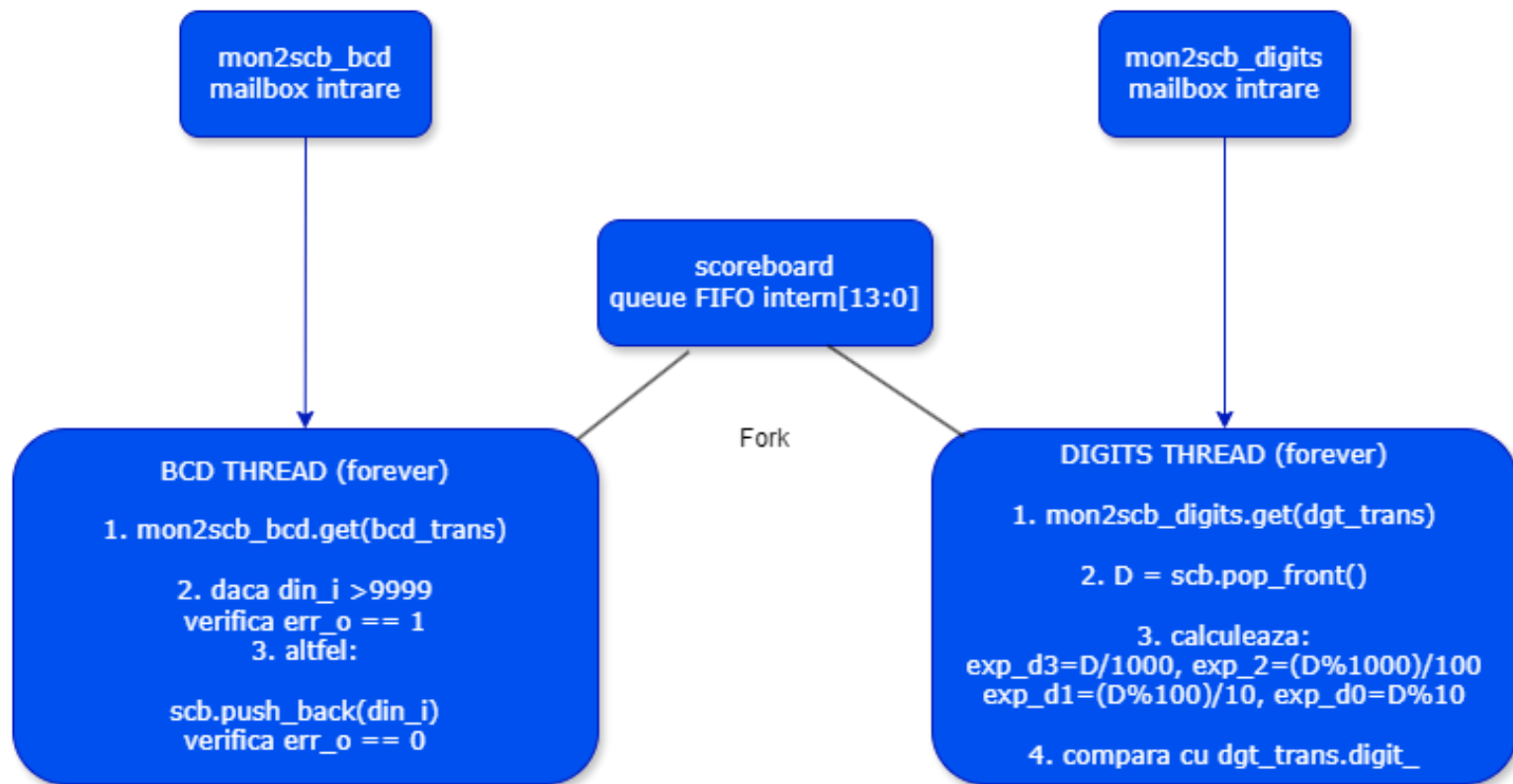
Comparatorul de date (scoreboard)

- Componenta decizională a mediului de verificare
- Conţine un FIFO intern (queue), care ţine valori valide trimise către DUT
- Rulează două fire de execuţie în paralel prin structura **fork/join**:
 - **BCD Thread** – preia tranzacţii de la monitorul BCD, verifică dacă avem valori valide şi erorile sunt tratate cum trebuie. Dacă avem valori valide -> introducem în FIFO intern
 - **Digits Thread** – preia tranzacţii de la monitorul Digits, extrage prima valoare din queue, calculează cifrele aşteptate prin împărţiri şi compară cu cifrele produse de DUT





Comparatorul de date (scoreboard)





Aserții

Aserțiile pe interfața de intrare

Aserție	Descriere
<code>assert_err_when_invalid</code>	Verifică că <code>err_o=1</code> când <code>valid=1</code> , <code>ready=1</code> și <code>din_i > 9999</code>
<code>assert_no_err_when_valid</code>	Verifică că <code>err_o=0</code> când <code>valid=1</code> , <code>ready=1</code> și <code>din_i <= 9999</code>
<code>assert_data_stable</code>	Verifică că datele rămân stabile cât timp <code>valid=1</code> și <code>ready=0</code> (caz de FIFO plin)
<code>assert_vld_hold</code>	Verifică că <code>valid</code> rămâne ridicat până când <code>ready</code> devine 1 (mecanism de retry)



Aserţii

Aserţiile pe interfaţa de ieşire

Aserţie	Descriere
assert_data_stable	Verifică că cifrele rămân stabile pe toată durata req=1, ack=0
assert_valid_bcd	Verifică că toate cifrele sunt în intervalul 0 - 9 (valorile 10 - 15 sunt ilegale)
assert_handshake_done	Verifică că req devine 0 în tactul următor de ceas, după ack=1
assert_ack_requires_req	Verifică că ack nu poate fi activ fără un req precedent sau curent
assert_req_start_clean	Verifică că ack este 0 la frontul crescător al req
assert_ack_follows_req_fall	Verifică că ack devine 0, după ce req cade



Scenarii de verificare şi teste implementate

Fişier	Scenariu	Descriere
bcd_random_test.sv	Test aleatoriu general	200 de tranzacţii aleatorii - valide şi invalide - cu delay BCD 1–10 şi delay ACK randomizat. Scenariul principal de verificare pentru acoperire generală.
test_valid_value.sv	Valoare validă singulară	0 singură tranzacţie cu bcd_din_i din [0:9999]. Verificare minimă - caz de bază pentru o singură conversie corectă.
test_invalid_value.sv	Valoare invalidă singulară	0 singură tranzacţie cu bcd_din_i din [10000:16383]. Verifică că err_o=1 şi că valoarea nu este introdusă în FIFO.
test_fifo_stress.sv	Stres FIFO full	80 de tranzacţii valide din [5000:9999] cu delay=1, primele 60 ACK la 20 tacturi. Simularea durează ~29ms (29095ns), demonstrând funcţionarea corectă sub back-pressure maxim susţinut.
test_fast_ack.sv	Burst cu ACK rapid	15 tranzacţii valide cu ACK instant (0–2 tacturi). Testează throughput-ul maxim, adică DUT-ul procesează cât mai rapid.
test_fast_input_slow_output.sv	Intrare rapidă, ieşire lentă	50 de tranzacţii cu delay BCD 1–2 tacturi şi ACK 50–200 tacturi. Stresează diferenţa de viteză între intrare şi ieşire - FIFO acţionează ca buffer.
test_b2b_trans.sv	Back-to-back	50 de tranzacţii consecutive cu delay=0 şi ACK instant (0–1 tacturi). Verifică aserţia p_vld_hold şi funcţionarea la viteză maximă.



Scenarii de verificare şi teste implementate

Fişier	Scenariu	Descriere
test_b2b_valid_invalid_valid sv	Back-to-back valoare validă -> invalidă -> validă	3 tranzacţii, una validă, una invalidă şi ultima validă cu delay = 0 şi ACK aleatoriu. Verifică comportamentul semnalului bcd_err_o şi comportamentul FIFO la valori invalide.
test_biggest_value sv	Cea mai mare valoare validă cu ACK rapid	10 tranzacţii valide ce conţin cea mai mare valoare validă (9999), transmise cu un delay mic (delay = 1) şi ACK instant. Verifică dacă toate cele 4 instanţe ale decodificării cifrelor funcţionează la capacitate maximă simultan.
test_smallest_value sv	Cea mai mică valoarea validă cu ACK rapid	10 tranzacţii valide ce conţin cea mai mică valoare validă (0), transmise cu un delay mic (delay = 1) şi ACK instant. Verifică dacă semnalele busy_o şi req_o funcţionează corect atunci când sărim peste etape de calcul.
test_reset sv	Reset în timpul descompunerii cifrelor	trimitem o tranzactie validă, aşteptăm ca busy_o să devină 1 - calcul în desfăşurare, iar după 2 tacte de ceas activăm reset; după reset verificăm dacă DUT-ul poate să proceseze o altă tranzacţie validă. Verificăm dacă DUT-ul se curăţă complet (registrele devin 0, iar FSM-ul intră în starea idle) şi nu rămâne blocat într-o stare nedefinită.
test_threshold_value sv	Transmitere valoare de prag dintre valid şi invalid	o singură tranzacţie, cu valoarea 10000. verificăm dacă comparatorul de eroare din Verilog foloseşte > sau >= corect
test_fifo_bounce sv	FIFO bounce - full -> empty -> full	6 tranzacţii valide - 1 pentru decodificare imediată + 5 pentru umplerea FIFO, transmise instant şi cu un ACK instant. ulterior, de 3 ori scoatem câte o valoare din FIFO şi adăugăm rapid una nouă. Verificăm dacă semnalele fifo_full şi bcd_rdy_o reacţionează fără erori.



Rezultatele testelor

Interfaţa	Element de acoperire funcţională	Intervalele declarate (bins)	Valoare maximă acoperită	Testul în care s-a obţinut valoarea maximă
BCD	intrare date (din_cp)	zero (0), valid_small [1-999], valid_mid [1000-4999], valid_large [5000-9999], invalid [10000-16383]	100%	bcd_random_test + test_smallest_value (pentru valoarea 0)
BCD	semnal eroare (err_cp)	Error low – 0, error high – 1	100%	bcd_random_test + test_invalid_value
Digits	Cifre ieşire (coverpoint_digit_3 -> coverpoint_digit_0)	Valori 0 – 9 pentru fiecare cifră, valori 10 – 15 invalide	100%	bcd_random_test
Digits	Delay ACK (coverpoint_delay)	no_delay (0), low_delay [1-10], medium_delay (>=11)	100%	test_fifo_stress + test_biggest_value (pentru no_delay)



Bug-uri găsite

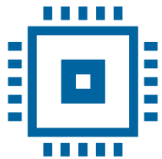
Numele bug-ului găsit	Testul în care l-am găsit	Observații
BUG_FIFO_FULL (fifo_full_o mereu 0)	test_fifo_stress.sv	Semnalul ready_o rămâne activ, deși FIFO este plin. Acest bug duce la pierderi de date prin suprascriere și deadlock din cauza desincronizării pointerilor.
BUG_OUTPUT_DATA (corupere date în 20% din cazuri)	bcd_random_test.sv	Au fost generate valori imposibile la ieşire, detectate de Scoreboard, Aserții şi Illegal bin hit în coverage.
BUG_RD_FROM_FIFO (nu citeşte ultimul element din FIFO)	test_fifo_bounce.sv	Contorul de citire sare peste ultima locație a memoriei (index – 4). Aceasta provoacă o acumulare de date necitite care corupe logica de empty/full, ducând la deadlock în handshaking.
BUG_THOUSANDS (digits_3 mereu 0)	test_b2b_trans.sv	Deși protocolul request/acknowledge funcționează corect, Scoreboard-ul detectează valori eronate doar pentru D3, confirmând un defect în prima etapă a algoritmului.

Converter BCD to Digits

Grigore Mihai-Alexandru – inginer verificare (input agent)

Cilichidreanu Andi-Marian – inginer verificare (output agent)

Ardeleanu Ștefan-Georgian – documentație tehnică



**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**